

2. 21世紀の正規化表としての構造化CSV：LHM、構文結合及びセマンティック結合

この文書は、UADCの共通データ層を概念面から説明します。構造化CSV（Structured CSV）は、一つの非正規化表ではなく、複数の論理的な正規化表を、疎な行とディメンション列によって一つの物理ファイルへ格納する方式です。

結合方式の全体像

構文結合（Syntax binding）

構文結合は、XMLなどの物理構文とLHMを対応付けます。Phase 1では、UBLの **XPath**、属性、条件及び **selector** を、LHMのクラス及び属性へ結び付けます。順方向ではXPathから値を取得し、逆方向では同じ結合表からXMLの要素及び属性を再構築します。

```
UBL XPath / selector / attribute
      ↑↓
    LHM semantic_path
      ↑↓
Structured CSV column
```

セマンティック結合（Semantic binding）

セマンティック結合は、元のXML構文を参照せず、LHMの **semantic_path** と構造化CSV列を、目的別の対象列へ対応付けます。セマンティックパスの末尾から元CSV列を導出し、クラス行の多重度から行スコープを決定します。

繰返しクラスを対象列へ横展開する場合は、次のようにゼロ始まりの **[n]** を使用します。

```
$.invoice.vatBreakdown[0].vatCategoryCode -> Tax1_Type_Code
$.invoice.vatBreakdown[1].vatCategoryCode -> Tax2_Type_Code
```

[n] は出力行を増やす指定ではなく、現在のインボイス又は行スコープ内のn番目の発生を選択する指定です。行を増やすかどうかは、インデックスを付けていないセマンティックパスと、最も深い繰返しクラスによって決まります。

現在の **semantic_binding.py** が解釈するセレクタは、ゼロ始まりの明示的な **[n]** 発生指定です。XPathの述語は評価しません。**[n]** だけで表せない業務条件は、セマンティック列、既定値、変換規則又は将来のセマンティック結合処理の拡張として定義する必要があります。

結合定義の責任分担

- LHMは、意味、階層、多重度及びデータ型の基準となる。

- 構文結合表は、XPath、selector、属性及び構文固有条件の基準となる。
- セマンティック結合表は、目的別の対象列、列順、**semantic_path**、****[n]****及び対応状況の基準となる。
- 構造化CSVは、Phase 1とPhase 2の間で構文に依存しない安定したインターフェースとなる。

データモデル及び支援ツールの詳細

1. 目的及びアーキテクチャ

この文書は、UADC Phase 1及びPhase 2で共通して使用するデータモデルを定義します。EN 16931 論理階層モデル（LHM）、構文結合表、構造化CSVの行、xBRL-CSVメタデータ及びタクソノミ並びにセマンティック結合表が、どのように連携するかを説明します。また、**tools/**以下の支援プログラムについて、同じ粒度で仕様を記載します。

アーキテクチャでは、次の三つの関心事を分離します。

- LHMは、安定したセマンティック識別子、階層、多重度及びデータ型を定義する。
- 構文結合は、UBL Invoiceなどの一つの元構文をLHMへ対応付ける。
- セマンティック結合は、LHMの値をADS PSV、ADS XBRL GL又はISO 21378 ADCなどのPhase 2対象へ対応付ける。

構造化CSVは、Phase 1とPhase 2の間で長期的に使用するインターフェースです。下流システムを元のXML構文へ依存させることなく、セマンティックな階層を保持します。

1.1 階層型Tidy Data

階層型Tidy Dataは、自然な構造に入れ子及び繰返し可能なクラスを含む情報へ、Tidy Dataの原則を拡張したものです。各セマンティック属性には一つの定義済み列があり、各発生を一度だけ表し、各セルには一つの値を格納します。さらに、ディメンション列によって、それぞれの疎な行がどのクラス発生に属するかを示します。

従来のフラットな表では、インボイスのヘッダー値及び取引当事者の値を、すべてのインボイス明細行へ繰返して格納します。この方法は、一つの矩形表として検索しやすい一方で、データを重複させ、同じファクトについて不整合な複製を生じさせる可能性があります。階層型Tidy Dataでは、インボイス、取引当事者、値引き、税及び明細の各発生を、それぞれが所有する行スコープに出力します。祖先ディメンションの値によって、すべての祖先属性を各子孫行へ複写することなく、これらの行を関連付けます。

生成される表は、意味上は階層的ですが、格納形式としてはTidyです。

- 各セマンティックファクトには、LHMによって決まる一つの列がある。
- 各クラス発生には、それを所有する一つの行スコープがある。
- 単一の子クラスは、最も近い繰返し祖先の行を共有する。
- 繰返し子クラスは、最初の発生から独立した行を使用する。
- 祖先及び子のディメンションによって、完全な階層コンテキストを識別する。
- 行スコープ外の空欄セルは、nullである業務ファクトの別の複製ではなく、**この行が所有していない**ことを意味する。

1.2 代替的な正規化形式としての構造化CSV

セマンティックなデータ交換では、構造化CSVを、**Coddの関係正規化**に代わる方法として提案します。Codd型の正規化は、データを複数の関係へ分解し、各ファクトを一度だけ格納し、主キー及び外部キーによって関係を接続することで、更新時の不整合を防ぎます。これはリレーショナルデータベース内では有効ですが、データ交換パッケージでは、複数の表、それぞれのキー及び結合規則をまとめて受け渡し、管理しなければなりません。

構造化CSVは、重複した複製を作らず、各ファクトの正本となる発生を一つだけ保持するという同じ本質的な目的を維持しながら、正規化された関係を一つの物理的な表で表します。本来は複数の正規化表となるものを、疎な行の所有関係及びディメンション列によって論理的に分離したまま、一つの表へまとめます。

関係正規化	階層型構造化CSV
エンティティ又は繰返しグループごとに一つの関係を定義する	繰返しLHMクラスごとに一つの行スコープを定義する
主キーが一つの関係内の行を識別する	ディメンション値がクラス発生を識別する
外部キーが子関係を親関係へ関連付ける	祖先ディメンション列が子行を親行へ関連付ける
結合によって複数の物理表を組み合わせる	一致するディメンションパスによって、一つのCSV内の階層を再構築する
親の値を子関係へ複写しない	親属性は親行に保持し、子行へ複写しない

したがって、**一つのCSVファイルが、一つの非正規化関係を意味するわけではありません**。一つのファイルは、論理的に正規化された複数の行スコープを格納するコンテナです。リレーショナル表を分離した場合に外部キーで表す関係を、ディメンションによって表現します。利用側は、各ディメンションが所有する行を選択して個別の論理表を再構築することも、祖先ディメンションのパスによって行をグループ化し、完全な階層を再構築することもできます。

このモデルは、リレーショナル正規形又はデータベース制約を否定するものではありません。長期的に使用でき、構文に依存しないデータ交換のために、異なる正規化及び直列化の方法を提供します。複数の論理表を、ファクトを重複させることなく一つのファイルへまとめ、関係を明示的に保持します。

1.3 正規化の不変条件

構造化CSVがこの正規化モデルに適合するためには、次のすべての条件を満たす必要があります。

1. セマンティック値は、そのLHMクラスを所有する行スコープだけに書き込む。
2. 子クラスが繰り返されるという理由だけで、親の値を繰り返さない。
3. 繰返し子の最初の発生を、親行へ統合しない。
4. 各子行は、親の発生を識別するために必要な祖先ディメンションを保持する。
5. ディメンション値は、その祖先スコープ内で一意である。
6. 元の階層は、物理的な行の隣接関係だけに依存せず再構築できる。

7. 構文結合及びセマンティック結合の処理は、双方向でこれらの所有関係及びディメンション規則を保持する。

第4章では、単一の子及び繰返し子に関する規範的な行パターンを示します。第8章及び第15章で説明する検証関数は、Phase 1及びPhase 2の処理中に、同じ不変条件を適用します。

2. EN 16931 LHM

2.1 元ファイル及び生成ファイル

LHMの元ファイル及び生成済み成果物は、**data/** 及び **out/phase1/** 以下で管理します。基準となるファイル名及び生成コマンドは、**01_ENVIRONMENT_TESTS_TUTORIAL.md** に記載しています。**tools/** 以下の生成プログラムは、元表を正規化し、セマンティックパスを導出し、変換プログラム及びタクソノミ生成プログラムが使用するファイルを作成します。

2.2 主要列

定義表及び結合表では、次の情報を使用します。

- ビジネス用語又はクラスのセマンティック識別子
- **C** がクラス、**A** が属性を示す型
- クラス階層を定義する親又はレベルの情報
- クラスが単一か繰返し可能かを定義する多重度
- 元構文に依存せず値を識別するセマンティックパス
- 構文固有のXPath、セクタ、データ型及び変換条件

識別子、階層及び多重度の組合せが基準です。インデント、行順及びラベルは可読性を高めますが、これらの構造列に代わるものではありません。

2.3 実効的な階層

クラス行が階層を構成します。属性行は、最も近い適用対象クラスに属します。単一の子クラスは、最も近い繰返し祖先の現在の行スコープへ統合します。繰返し子クラスは、最初の発生から独自の行スコープを作成します。この規則は、結合定義の生成、構造化CSVの作成及び逆変換で共通して使用します。

2.4 命名及び安定性

セマンティック識別子及び確立済みの名前空間識別子は、構文の改訂をまたいで安定させます。データの同一性を変更しない限り、人が読むためのラベルは改善できます。生成済みタクソノミのエントリーポイントには、**en16931-oim-2026-07-05.xsd** のように、日付を含むファイル名を使用します。

3. セマンティックパス、XPath及び結合定義の基準

3.1 セマンティックパス

セマンティックパスは、LHM階層内のクラス又はビジネス用語を識別します。Phase 2変換で使用する構造化CSVの元列及び繰返し行スコープを決定するために使用します。そのため、Phase 2の変換プログラムは、元UBLのXPath規則を再構築しません。

3.2 構文XPath

構文結合表は、セマンティックパスを元構文又は対象構文のXPathへ対応付けます。順方向変換では、XML文書に対して元XPathを評価します。逆変換では、文書ルートから絶対XPathを構築し、欠けている祖先を文書順に作成します。絶対パスとして処理することで、**AccountingSupplierParty**などの要素が、**PaymentMeans**内に存在する入れ子の**Invoice**の下へ誤って作成されることを防ぎます。

3.3 結合定義の基準

適用する結合表は、構文固有のセレクト、属性、条件及び発生処理について基準となります。LHMは、セマンティック階層及び多重度について引き続き基準となります。生成する統合表は、どちらか一方を暗黙に選ぶのではなく、両方の基準を保持し、競合がある場合は検証時に明らかにしなければなりません。

4. 構造化CSVの階層

4.1 ディメンション列

構造化CSVでは、繰返しクラスごとに一つのディメンション列を使用します。ディメンション値は、現在の祖先スコープ内における一つの発生を識別します。属性列には値を格納し、空欄セルは、その行が当該属性を所有していないことを示します。

4.2 単一の子クラス

繰返し **dAaa** の下で **dBbb** が単一である場合、両クラスの値は同じ行を共有します。

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,b1V1,b2V1,b3V1
```

単一の子は、最も近い繰返し祖先の行に属し、独立した発生行を持ちません。

4.3 繰返し子クラス

dBbb が繰返し可能である場合、親属性は親行に格納し、最初の発生を含む各子発生を独立した行に格納します。

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,

```

```
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

次の圧縮形式は、最初の繰返し子を親行へ統合しているため、無効です。

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,1,a1V1,a2V1,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

4.4 所有関係の導出

変換プログラムは、**type=C** の結合行、クラスの **multiplicity** 及び各属性の **semantic_path** から、列の所有関係を導出します。単一の子は、最も近い繰返し祖先の行スコープを継承します。繰返し子は新しいディメンションを所有し、発生番号1から独立した行を出力します。この規則を、すべての深さで再帰的に適用します。

4.5 疎な行の仕様

行は意図的に疎な構成とします。親行には、そのスコープが所有する値だけを格納します。繰返し子行には、その祖先を識別するために必要なディメンションと、その子スコープが所有する値を格納します。Phase 2の統合処理では、対象結合が必要とする場合に限り、現在の祖先値を対象レコードへ引き継ぎます。

5. xBRL-CSVタクソノミモデル

5.1 エントリーポイント及び定義リンクベース

OIMエントリーポイントは、**en16931-oim-2026-07-05.xsd** です。その定義リンクベースは、**en16931-def-2026-07-05.xml** です。日付を含む名前によって、この生成済みタクソノミー式を、以前のタプル指向又はOIM非対応のエントリーポイントと区別します。

5.2 タクソノミの構成要素

OIMタクソノミには、次のものが含まれます。

- 概念スキーマ及びディメンションスキーマをimportするエントリーポイントスキーマ
- LHMビジネス用語の概念
- 繰返しクラスの発生を表す明示的ディメンション
- xBRL-CSVコンテキストで必要となるドメイン及びメンバーの宣言
- ハイパーキューブ、ディメンション、ドメイン及び適用対象概念を接続する定義リンクベース
- 元LHMから提供される場合は、ラベル及び参照

UADC OIMタクソノミでは、タプルを使用する必要はありません。階層は、ディメンション及び定義関係によって表現します。

5.3 ディメンションの導出

繰返し可能な各LHMクラスを、一つのディメンションスコープとします。単一の子孫は、そのスコープ内に残ります。入れ子の繰返し可能クラスは、祖先ディメンションを保持したまま、新しいディメンションを追加します。これは、第4章の構造化CSV所有規則と対応しており、Arelleから意図したディメンション関係を確認できます。

6. 通貨、セレクト及び条件付きファクト

6.1 通貨コード

通貨識別子は、**Currency.csv** を含む該当コードリスト表によって正規化します。金額列には数値を保持し、結合定義又はxBRL-CSVメタデータによって、対応する通貨単位を指定します。

6.2 BT-110及びBT-111

BT-110 Invoice total VAT amount は、金額の通貨コードが文書通貨コードと一致する場合に選択します。**BT-111 Invoice total VAT amount in accounting currency** は、金額の通貨コードが税務会計通貨コードと一致する場合に選択します。この条件は、金額要素の通貨識別子と文書レベルの通貨フィールドを併せて使用し、要素の出現順だけに依存してはなりません。

Allowanceサンプルでは、構造化CSVは二つの条件付き値を別々のセマンティック列に保持しなければなりません。文書通貨のVAT合計は **1225.00**、会計通貨のVAT合計は **9324.00** です。条件付きファクトが空欄になる場合は、結合、セレクト又は通貨比較に不具合があることを示すため、回帰テストで確認します。

6.3 売手住所のセレクト

ADS結合では、識別子型 **V** によって売手の識別情報を選択します。関連する住所は、そのセレクトの下にある **identifierAddress** に格納します。Phase 2 ADS XBRL GLの売手一覧へ売手住所を出力するため、統合表の生成時に、このセレクトコンテキストを保持しなければなりません。

7. 監査の継続性及びデータの境界

ADS及びADCは、通常、個別のインボイスとは別に、ユーザー操作、会計明細、締め仕訳及び年次報告書作成を記録するERP環境で使用します。そのため、インボイスだけでは、対象モデルのすべての監査証跡、転記、承認又は報告フィールドを設定できません。

UADCは、この境界を明示します。インボイスから取得できるすべての情報を対応付け、設定できない対象フィールドを既知の不足項目として保持し、独立して設計されたインターフェースファイル間で、安定したセマンティック識別子及び階層を維持します。これにより、組織は独自の入出力形式を維持しながら、LHM及び結合機構を長期的なセマンティック層として使用できます。ERPの操作履歴及び管理された結合定義の版と組み合わせることで、長期にわたり再現可能で、監査に利用できるデータ保存の基盤となります。

8. 支援ツール仕様

8.1 目的

この章は、**tools/** 以下のすべての支援プログラムを規定します。各ツールについて、役割、入力、出力、コマンドラインインターフェース、処理順序、主要な関数及びデータ構造、検証動作並びに運用上の制約を、同じ粒度で記録します。

これらのツールは、次の四つの活動を支援します。

- EN 16931 LHMの構築及び保守
- 構文結合の準備及びチュートリアル変換
- タクソノミ、メタデータ及びラウンドトリップ成果物の生成
- 生成したフラットファイルの確認

運用用のPhase 1及びPhase 2変換プログラムは、**src/** 以下にあります。**tools/** 以下のプログラムは、定義及びテスト成果物を準備するか、学習用の小規模な実装を提供します。

8.2 適用範囲

この仕様は、次の15プログラムを対象とします。

No.	プログラム	分類
1	audit_en16931_coverage.py	LHM対応範囲の監査
2	build_lhm_from_source.py	LHM生成
3	build_roundtrip_test_artifacts.py	テスト成果物の生成
4	build_syntax_binding.py	結合定義の生成
5	check_lhm_class_element.py	LHM検証
6	extend_en16931_lhm_coverage.py	LHM保守
7	normalize_lhm_class_element.py	LHM正規化
8	normalize_lhm_semantic_paths.py	LHM正規化
9	order_lhm_by_en16931_table2.py	LHMの順序付け
10	psv_viewer.html	ブラウザでの確認
11	tutorial/semantic_binding_sample.py	セマンティック変換のチュートリアル
12	tutorial/syntax_binding_sample.py	構文変換のチュートリアル
13	taxonomy/xBRLGL_TaxonomyGenerator.py	タクソノミ生成
14	update_lhm_definitions_from_pdf.py	LHM定義の更新
15	update_lhm_syntax_sequence_from_ubl_xsd.py	LHM構文順の更新

8.3 共通規則

8.3.1 実行ディレクトリ

この文書に示すコマンドは、UADC PoCリポジトリのルートディレクトリで実行します。

Windows PowerShell :

```
$python = 'python'
& $python .\tools\PROGRAM.py [arguments]
```

macOS又はLinux :

```
PYTHON=python3
$PYTHON ./tools/PROGRAM.py [arguments]
```

8.3.2 CSVの文字エンコーディング

文字エンコーディングのオプションを持つツールでは、既定値を **utf-8-sig** とします。これは、表計算ソフトウェアとの互換性のため、UTF-8のバイトオーダーマークを保持します。文字エンコーディングのオプションを持たないツールも、LHM CSVを **utf-8-sig** で読み書きします。

8.3.3 終了ステータス

通常は、次の規則を使用します。

- 終了ステータス **0** : 処理又は検証に成功した。
- 終了ステータス **1** : 処理済みの検証エラー又は変換エラーが発生した。
- 捕捉されない例外 : 入力ファイル、スキーマ、必須列又は依存パッケージが不足しているか、無効である。

8.3.4 入力ファイルの直接更新

次のプログラムは、入力LHM CSVを直接更新します。

- **extend_en16931_lhm_coverage.py**
- **normalize_lhm_class_element.py**
- **order_lhm_by_en16931_table2.py**
- **update_lhm_definitions_from_pdf.py**

normalize_lhm_semantic_paths.py も、**--output** を省略した場合は入力ファイルを直接更新します。これらのツールを実行する前に、元ファイルを確認するか、複製してください。

8.4 ツール仕様

8.4.1 audit_en16931_coverage.py

役割

標準PDFで検出したすべてのEN 16931 **BG-n** 及び **BT-n** 識別子が、LHM CSVに含まれているかを監査します。

入力及び出力

- 入力1：EN 16931-1 PDF
- 入力2：id 列を含むLHM CSV
- 出力：標準出力へ書き出すJSONレポート
- 副作用：なし

コマンド

```
& $python .\tools\audit_en16931_coverage.py STANDARD.pdf LHM.csv
```

処理

1. **extract_pdf_ids** が、**pypdf.PdfReader** を使用してPDFの全ページを読み込む。
2. 正規表現 **BG-n** 又は **BT-n** によって、各識別子が最初に現れるページを収集する。
3. **read_lhm_ids** が、LHMの **id** 列から該当する識別子を収集する。
4. 不足する識別子及び余分な識別子の集合を計算する。
5. 件数、PDFのページ番号を付けた不足識別子及び余分なLHM識別子を、整形済みJSONとして出力する。

関数

関数	責務
sort_key	BG又はBTの接頭辞及び数値接尾辞によって識別子を並べ替える。
extract_pdf_ids	PDFから識別子と最初のページの対応を抽出する。
read_lhm_ids	LHMから有効なBG及びBT識別子を読み込む。
main	入力を解析し、集合を比較し、JSONを出力してステータスを返す。

終了状態及び依存関係

PDFの識別子が一つでもLHMに不足する場合は **1**、それ以外は **0** を返します。**pypdf** が必要です。
LHMに余分な識別子がある場合は報告しますが、それだけではエラーとしません。

8.4.2 build_lhm_from_source.py

役割

編集可能なEN 16931元表と、変換及びタクソノミ生成スクリプトが使用する正規化済みLHM CSVを分離して管理します。

入力及び出力

init-source コマンドは、生成済みLHMを読み込み、編集可能な元CSVを書き出します。**build** コマンドは、その編集用元CSVを読み込み、型、多重度、階層レベル、セマンティックパス、クラス用語及び一意の要素名を正規化したLHMを生成します。

コマンド

```
& $python .\tools\build_lhm_from_source.py init-source LHM.csv SOURCE.csv
& $python .\tools\build_lhm_from_source.py build SOURCE.csv LHM.csv
```

処理

init-source では、**source_from_lhm** がレビュー対象の業務フィールドを複写し、上書き用の列を作成します。**build** では、次の処理を行います。

1. 元データ行を **sequence** 順に並べ替える。
2. 型が設定されていない場合、**type_from_id** がクラス又は属性の型を導出する。
3. **normalize_multiplicity** が元のカーディナリティを、対応するLHM表記へ変換する。
4. **semantic_path** は、明示的な上書き値を使用するか、**path** 識別子に従ってlowerCamelCaseのセグメントを組み立てる。
5. **nearest_bg_id** が、各ファクトのセマンティッククラスを選択する。
6. **lhm_effective_levels** が、インボイスルート、繰返しクラス及び最も近い実効的な繰返し祖先が所有するファクトにだけレベルを割り当てる。
7. 明示的なクラス用語及び要素の上書き値を保持し、不足する要素は **unique_element_names** で生成する。
8. LHMを書き出す前に、要素名の重複及び未対応の多重度をエラーとする。

関数

関数	責務
normalize_multiplicity	カーディナリティ表記を正規化する。
read_rows 、 write_rows	UTF-8 BOM付きCSVレコードを読み書きする。
type_from_id	BGをクラス行、BTを属性行へ対応付ける。
lower_camel_case_concatenated	セマンティックパスのセグメントを生成する。
source_from_lhm	編集可能な元形式を作成する。
path_ids	一つの行について階層識別子を返す。

関数	責務
semantic_path	セマンティックパスを構築するか、上書き値を適用する。
nearest_bg_id	所有するビジネスグループを検出する。
lhm_effective_levels	実効的な階層型CSVレベルを計算する。
build_lhm	生成済みLHMの構築を統括する。
validate_unique_elements	概念名の重複をエラーとする。
validate_multiplicities	未対応の多重度をエラーとする。
main	init-source 又は build を実行する。

検証

生成後は、LHMセマンティックパス、階層レイアウト及びタクソノミ生成プログラムの回帰テストを実行してください。生成済みLHMが基準であり、通常、手作業による変更は編集用の元CSV又は上書き列に対して行います。

8.4.3 build_roundtrip_test_artifacts.py

役割

回帰テストで使用するPhase 1ラウンドトリップ成果物一式を再構築します。

入力及び出力

入力は **DATASETS** に固定されています。

- 最小OpenPeppol UBL Invoiceサンプル
- BIS Billing 3サンプルディレクトリ内のすべてのUBL Invoice XMLファイル

各データセットについて、次のディレクトリを再作成します。

```
tests/roundtrip/DATASET/source_xml/
tests/roundtrip/DATASET/structured_csv/
tests/roundtrip/DATASET/metadata_json/
tests/roundtrip/DATASET/roundtrip_xml/
```

コマンド

```
& $python .\tools\build_roundtrip_test_artifacts.py
```

このプログラムにはコマンドラインオプションがありません。引数解析を使用していないため、`--help` を指定しても生成処理を実行します。

処理

- 1. `ensure_taxonomy` がOIMエントリースキーマ及びEN 16931モジュールスキーマを確認し、存在しない場合はタクソノミ生成の回帰テストスクリプトを呼び出す。
- 2. `is_invoice_xml` が、ルート要素がInvoiceでないXML文書を除外する。
- 3. `clean_directory` が、対象となる各成果物ディレクトリ内の既存ファイルを削除する。
- 4. `build_dataset` が元XMLを複写し、構造化CSV及びメタデータJSONを生成し、そのCSVをUBL XMLへ逆変換する。
- 5. `run_converter` が、現在のPython実行ファイルを使用して `src/syntax_binding.py` を呼び出し、子プロセスが失敗した場合は直ちに停止する。

関数

関数	責務
<code>ensure_taxonomy</code>	必要な生成済みタクソノミスキーマが存在することを確認する。
<code>is_invoice_xml</code>	XMLルートのローカル名を確認する。
<code>clean_directory</code>	一つの成果物ディレクトリを再作成し、空にする。
<code>run_converter</code>	順方向又は逆方向の構文結合を実行する。
<code>build_dataset</code>	一つのサンプル集合について4種類の成果物を作成する。
<code>main</code>	設定済みの二つのデータセットを生成し、合計件数を報告する。

制約

このツールは再生成用です。再構築前に、四つの対象成果物ディレクトリ直下の既存ファイルを削除します。生成後は、ラウンドトリップ成果物、UBLスキーマ及びxBRL-CSVメタデータの検証スクリプトを実行してください。

8.4.4 build_syntax_binding.py

役割

`semantic_path` と `xpath` の組を持つ単純な表から、簡易結合CSV又はLHM／HMD形式の構文結合を生成します。

入力及び出力

- 入力：`semantic_path` 及び `xpath` の両方を含むCSV
- 出力：`--output` で指定した結合CSV

- 既定の文字エンコーディング : **utf-8-sig**

コマンド

```
& $python .\tools\build_syntax_binding.py BINDINGS.csv `
  --output SYNTAX_BINDING.csv [--simple] [--encoding utf-8-sig]
```

--simple を指定しない場合、完全なLHMヘッダーを持つ生成済みクラス行及び属性行を出力します。**--simple** を指定した場合は、**column**、**semantic_path** 及び **xpath** だけを出力します。

処理

1. **read_source_rows** が必須列を検証し、不完全な組をエラーとする。
2. **split_semantic_path** が、述語の角括弧外にあるピリオドでパスを分割する。
3. **clean_token** 及び **semantic_path_to_column** が、安全で安定した列名を作成する。
4. **unique_name** が、重複する名前へ数値接尾辞を追加する。
5. **build_simple_rows** が入力の組ごとに一行を出力するか、**build_lhm_rows** が不足する祖先クラス行と、それに続く属性行を作成する。
6. **build_syntax_bindings** が出力ディレクトリを作成し、行を書き出す。

関数

関数	責務
semantic_path_to_column	セマンティックパスから対象列を導出する。
split_semantic_path	述語を保持したまま、パスセグメントを分割する。
segment_label 、 segment_element	ラベル及び要素トークンを導出する。
clean_token 、 unique_name	識別子を正規化し、重複を解消する。
read_source_rows	元の組を検証する。
build_simple_rows	簡易形式を生成する。
build_lhm_rows	クラス行及び属性行を生成する。
build_syntax_bindings	選択した出力形式を書き出す。
main	引数を解析し、処理済み例外を終了ステータス 1 へ変換する。

制約

生成するクラスの多重度は **1..***、生成するファクトの多重度は **0..1**、生成するファクトのデータ型はTextです。実運用の結合定義として使用する前に、これらの既定値を確認してください。

8.4.5 check_lhm_class_element.py

役割

CSVを変更せずに、LHMの **class_term** 及び **element** の値が正規化規則に一致するかを検証します。

コマンド及び入力

```
& $python .\tools\check_lhm_class_element.py LHM.csv
```

処理

1. LHMを読み込み、識別子と名称の対応を作成する。
2. **normalize_lhm_class_element.py** の **nearest_parent_bg**、**singularize** 及び **unique_element_names** を再利用する。
3. BG行及びBT行について、想定値と格納値を比較する。
4. 各分類について最大50件の不一致と、総件数を出力する。

関数

関数	責務
read_rows	正規化済みLHMの辞書を読み込む。
main	想定値を計算し、差異を報告してステータスを返す。

どちらかの不一致一覧が空でない場合は **1** を返します。それ以外の場合は、確認した行数を出力し、**0** を返します。

8.4.6 extend_en16931_lhm_coverage.py

役割

既存のLHM CSVに対し、プログラムで管理しているEN 16931の対象範囲に関する追加及び修正を適用します。

コマンド及び副作用

```
& $python .\tools\extend_en16931_lhm_coverage.py LHM.csv
```

入力ファイルを直接書き換えます。

処理

1. 静的データ **UPDATES** により、識別子が既知の行を修正する。
2. 静的データ **ROWS** により、不足しているBG及びBTの行を定義する。
3. 既存の識別子は重複して追加しない。
4. **write_rows** により **sequence** を振り直し、**path** から **level** を再計算する。

関数及びデータ

項目	責務
class_row 、 attr_row 、 row	正規化済みの追加レコードを生成する。
UPDATES	既存識別子に対する項目修正を定義する。
ROWS	存在しない場合に追加するレコードを定義する。
read_rows 、 write_rows	LHM全体を読み込み、書き戻す。
main	修正及び追加を適用し、追加件数を報告する。

制約

このプログラムはデータ駆動型ですが、汎用的なPDF解析プログラムではありません。実行後は、派生項目の整合性を維持するため、並べ替え、セマンティックパスの正規化、クラス名及び要素名の正規化並びにLHMの各種検査を実行してください。

8.4.7 normalize_lhm_class_element.py

役割

LHMの **class_term** 及び **element** を正規化し、入力ファイルを直接更新します。

コマンド

```
& $python .\tools\normalize_lhm_class_element.py LHM.csv
```

規則

- BG行は自分自身のグループに属し、BT行は最も近い祖先BGに属する。
- **class_term** には、所属するBGのビジネス用語を単数形にした名称を使用する。
- **element** は大文字で始め、**semantic_path** の末尾から重複しない最短の部分を使って生成する。
- 名称が重複する場合は、セマンティックパスの上位セグメントを順に追加して区別する。

関数

関数	責務
singularize	英語の単数形変換規則を適用する。
semantic_segments	セマンティックパスを名称セグメントに分割する。
upper_camel	一つのセグメントをUpperCamelCaseへ変換する。
suffix_element_name	パス末尾のセグメントから要素名を生成する。
unique_element_names	各行について、重複しない最短の要素名を決定する。
nearest_parent_bg	所属する最も近いBG識別子を取得する。
read_rows 、 write_rows	入力CSVを読み込み、書き戻す。
main	変更対象セルを更新し、変更した値の件数を報告する。

正規化後は、直ちに **check_lhm_class_element.py** を実行してください。

8.4.8 normalize_lhm_semantic_paths.py

役割

行の名称及び識別子階層から、LHMのすべての **semantic_path** を再構築します。

コマンド

```
& $python .\tools\normalize_lhm_semantic_paths.py INPUT.csv `
  [--output OUTPUT.csv] [--encoding utf-8-sig]
```

--output を省略した場合は、**INPUT.csv** を置き換えます。

処理

1. **lower_camel_case_concatenated** により、各ビジネス用語を一つのパスセグメントへ変換する。空の用語には **unnamed** を使用し、数字で始まる名称には接頭辞を付ける。
2. **normalize_rows** により、行識別子とセグメントを対応付ける。
3. 各行について、**path** に示された順序でセグメントを選択する。

- 4. 結果を `$.segment.child` 形式で書き込む。
- 5. `normalize_file` は、元のCSV列順を維持する。

関数

関数	責務
<code>lower_camel_case_concatenated</code>	<code>lowerCamelCase</code> のセグメントを生成する。
<code>normalize_rows</code>	メモリ上のセマンティックパスを置き換える。
<code>normalize_file</code>	CSV全体を読み込み、正規化して書き出す。
<code>main</code>	出力先を決定し、エラーを処理する。

この単独実行ツールは、明示的なセマンティックパスの上書き定義を参照しません。編集可能なソースに定義した上書きを保持する必要がある場合は、`build_lhm_from_source.py` を使用してください。

8.4.9 `order_lhm_by_en16931_table2.py`

役割

組み込まれたEN 16931-1 表2の識別子順に従ってLHMの全行を並べ替え、既知の階層修正を適用します。

コマンド及び副作用

```
& $python .\tools\order_lhm_by_en16931_table2.py LHM.csv
```

入力ファイルを直接書き換えます。

処理

- 1. 並べ替え前に、**FIXES** により既知の行項目を修正する。
- 2. **TABLE2_ORDER** に、請求書ルートから始まる完全な識別子順を定義する。
- 3. 順序一覧に存在しない識別子があればエラーとする。不明な行を末尾へ暗黙に配置することはない。
- 4. 行を並べ替え、**sequence** を振り直し、**path** から **level** を再計算する。

関数及びデータ

項目	責務
<code>TABLE2_ORDER</code>	プログラム上の基準となる順序を定義する。

項目	責務
FIXES	階層又は項目に関する既知の修正を適用する。
read_rows、write_rows	レコードを読み込み、並べ替え後に書き戻す。
main	順序一覧の完全性を検証し、並べ替えて行数を報告する。

8.4.10 psv_viewer.html

役割

生成されたADS PSV、CSV、タブ区切り又はテキストファイルを閲覧するための、単独で動作するブラウザビューアです。データはアップロードされず、Webサーバも必要ありません。

ユーザー入力及び出力

- 入力：ページで選択又はドラッグ&ドロップしたローカルファイル。
- 区切り文字：パイプ、カンマ又はタブ。
- 絞り込み：表示行に対して適用する、大文字・小文字を区別しない文字列。
- 出力：ブラウザ内だけに表示される対話型HTML表。

処理

1. **loadFile** が **FileReader** で選択ファイルを読み込み、BOMを除去する。
2. **normalizedDelimiter** が画面上の選択肢を実際の区切り文字へ変換する。
3. **parseDelimited** が、引用符付きセル、エスケープされた引用符、区切り文字及び改行を、サーバ側ライブラリを使わずに解析する。
4. **renderTable** が先頭行を見出しとして扱い、行番号を追加し、全データ行で空の列を非表示にする。
5. **applyFilter** が、小文字化した検索対象文字列に基づいて行の表示・非表示を切り替える。
6. **updateMeta** が、ファイル名、行数、列数、非表示列及び表示中の行数を表示する。

関数

関数	責務
parseDelimited	引用符処理を含む区切りテキストを解析する。
normalizedDelimiter	パイプ、カンマ又はタブを解決する。
renderMessage	空ファイル又は警告メッセージを表示する。
renderTable	見出し固定のHTML表を生成する。
updateMeta	ファイル及び表の統計情報を更新する。
applyFilter	ファイルを再解析せずに行を絞り込む。

関数**責務****loadFile**

ローカルファイルの読み込みと表示を統括する。

制約

このビューアは内容確認用であり、検証又は編集を行うものではありません。すべての処理はブラウザのメモリ上で行われ、ページを閉じるか再読み込みすると、読み込んだデータは失われます。

8.4.11 tutorial/semantic_binding_sample.py**役割**

構造化CSVからフラットCSVへ、入力行と出力行を一対一で対応させる簡易なセマンティック結合を例示します。これは運用用のPhase 2変換プログラムではありません。実運用では、クラスの多重度、繰返し行スコープ、添字付きセマンティックパス、出力形式及びディレクトリ入力に対応する **src/semantic_binding.py** を使用します。

コマンド

```
& $python .\tools\tutorial\semantic_binding_sample.py STRUCTURED.csv `
--binding BINDING.csv --output FLAT.csv [--encoding utf-8-sig]
```

結合定義の互換性

読み込み処理は、出力項目名、入力パス及び固定値について、旧版の代替列名も受け付けます。スラッシュ又はドットで区切られた入力パスの最後のセグメントを、構造化CSVの列名として使用します。

処理

1. **read_bindings** が、出力項目名を持ち、入力又は固定値のいずれかが指定された行を選択する。
2. **tail** が入力パスを列名へ短縮する。
3. 入力構造化CSVの各行を、互いに独立した行として読み込む。
4. 各入力行について、各出力項目に入力セルの値又は固定値の代替値を設定する。
5. 入力行ごとに一つの出力行を書き出す。

関数

関数	責務
first_present	旧版の候補列名から、最初に値が設定されているものを取得する。
tail	パスの最後のセグメントを取り出す。
read_bindings	使用可能な結合定義行を正規化する。
convert_structured_to_flat	行単位の射影変換を実行する。
main	引数を解析し、入出力の行列数を報告してエラーを処理する。

制約

このチュートリアルは、階層行の統合、繰り返される祖先の推定、又は構造化CSVにおける単一クラスと繰り返し子クラスの規則を処理しません。

8.4.12 tutorial/syntax_binding_sample.py

役割

XMLからCSVへの小規模な構文結合実装を例示します。これは運用用のPhase 1変換プログラムではありません。実運用の順方向変換及び逆方向変換には **src/syntax_binding.py** を使用します。

コマンド

```
& $python .\tools\tutorial\syntax_binding_sample.py INPUT.xml `
--binding BINDING.csv --output OUTPUT.csv `
[--row-xpath XPATH] [--encoding utf-8-sig]
```

処理

1. **collect_namespaces** が入力XMLの名前空間宣言を読み込む。
2. **read_bindings** が、複数の旧版結合定義列名を受け付ける。
3. **find_nodes** が、子要素ステップに限定したXPath評価を行う。
4. **--row-xpath** を指定した場合は、繰り返し行のコンテキストを選択する。省略した場合は、文書ルートから一行を生成する。
5. 各結合定義が要素のテキスト又は末端属性を抽出する。複数のテキストが一致した場合は、パイプで連結する。
6. 抽出結果が空の場合は、結合定義の既定値を使用する。

対応する述語

限定的なXPath評価器は、次の述語に対応します。

- 子パスと引用符付きリテラルの等価又は非等価比較
- 絶対文書パスとの等価又は非等価比較
- 子パスと **true()** 又は **false()** の比較
- 末端属性及び名前空間接頭辞付き要素ステップ

未対応の述語はエラーにならず、現在は一致したものとして扱われます。この点は、このサンプル変換プログラムを運用上の基準として使用してはならない重要な理由です。

関数

関数群	責務
local_name 、 collect_namespaces 、 qualify_step	名前空間を処理する。
split_step_predicate 、 split_xpath 、 split_terminal_attribute	XPathを解析する。
path_value 、 predicate_matches 、 child_matches	述語を評価する。
find_nodes 、 get_value	XMLノード及び単一値を抽出する。
first_present 、 read_bindings	旧版結合定義を正規化する。
write_structured_csv	選択したコンテキストごとに一行を出力する。
main	引数を解析し、変換エラーを処理する。

8.4.13 taxonomy/xBRLGL_TaxonomyGenerator.py

役割

LHM/HMD形式の定義CSVから、ローカルのXBRL-CSVタクソノミ、ディメンション定義関係、表示関係、ラベル、メタデータひな型及びCSVスケルトンを生成します。

コマンド

```
& $python .\tools\taxonomy\xBRLGL_TaxonomyGenerator.py LHM.csv `
  --base_dir out\taxonomy `
  --palette en16931 `
  --root invoice `
  --lang en `
  --currency JPY `
  --namespace http://www.xbrl.org/int/gl/plt/2026-07-05 `
  [--encoding utf-8-sig] [--trace] [--debug]
```

引数

引数	意味
inFile	入力するLHM/HMD CSV。
--base_dir	生成するタクソノミのルート。既定値は現在のディレクトリ。
--palette	レコードにモジュールがない場合に使用する既定モジュール。
--root	メタデータ列に使用するルートセマンティック識別子。
--lang	ローカルラベルの言語。既定値は ja 。
--currency	金額列に使用するISO 4217通貨単位。既定値は JPY 。
--namespace	タクソノミ名前空間。末尾10文字をバージョンとして使用する。
--encoding	CSV及び生成テキストの文字エンコーディング。
--trace、--debug	診断出力を制御する。

処理

1. コンストラクタが入力を検証し、出力ルートを作成する。
2. **load_csv_data** がUADC LHMの見出しを受け付け、レコードを正規化し、データ型を対応付け、表示上の親を決定し、**lhm_level** と多重度から有効なディメンション所有関係を構築する。
3. タクソノミ出力には、ファクト及び有効なディメンションクラスだけを残す。
4. **process_records** が繰り返しクラスの親関係及びロール情報を決定する。
5. **generate_taxonomy_files** がモジュールスキーマ、OIMエントリスキーマ、表示関係、ラベル及びディメンション定義関係を書き出す。
6. **ensure_gl_gen_schema** が汎用データ型スキーマを検出し、又はバージョンを整える。
7. ルートが指定されている場合、**json_meta_file** がxBRL-CSV JSONひな型及びCSVスケルトンを書き出す。

主な生成ファイル

EN 16931プロファイル、バージョン **2026-07-05** の主な生成ファイルは次のとおりです。

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
out/taxonomy/plt/en16931-def-2026-07-05.xml
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/xbrl-gl.json
out/taxonomy/xbrl-gl_skeleton.csv
```

クラスメソッド

メソッド	責務
init	パス、バージョン、オプション及び順序付きモデルマップを初期化する。
debug_print 、 trace_print 、 error_print	診断及び致命的エラーを出力する。
ensure_gl_gen_schema 、 gl_gen_schema_location	汎用データ型スキーマを提供する。
concept_item_type	LHMのデータ型をXBRL項目型へ対応付ける。
LC3 、 titleCase 、 SC 、 escape_text	名称及びテキストを正規化する。
getRecord 、 getParent 、 getChildren 、 getElementID	モデルを参照する。
domainMember 、 defineHypercube	ディメンション関係を構築する。
roleRecord 、 linkPresentation	ロール及び表示関係を構築する。
normalize_lhm_record	UADC及び旧版LHMの列を適応させる。
load_csv_data	レコードを読み込み、表示及びディメンションのマップを構築する。
process_records	繰返しクラス及びロールの関係を確定する。
generate_taxonomy_files	スキーマ及びリンクベースを書き出す。
json_meta_file	メタデータ及び任意のCSVスケルトンを書き出す。

検証及び制約

tests/test_xbrlgl_generator_uadc_lhm.py 及び **tests/validate_taxonomy.py** を実行し、その後 Arelle でメタデータを検証してください。OIM エントリポイントは **en16931-oim-2026-07-05.xsd**、ディメンション定義リンクベースは **en16931-def-2026-07-05.xml** です。タプル/コンテンツ用のエントリスキーマは、このPoCプロファイルの対象外です。

8.4.14 update_lhm_definitions_from_pdf.py

役割

EN 16931-1 表2のDescription列から、LHMの **definition** を更新します。

コマンド


```
& $python .\tools\update_lhm_definitions_from_pdf.py STANDARD.pdf LHM.csv  
、  
[--first-page 43] [--last-page 75]
```

LHMを直接書き換えます。ページ番号は1から始まります。

処理

- 1. **extract_descriptions** が **pdfplumber** を使い、指定範囲のページにある表を調べる。
- 2. **clean_identifier** が先頭セルからBG又はBTの識別子を取り出す。
- 3. **clean_description** が空白を正規化し、Descriptionという見出しを除去する。
- 4. 複数の表行にまたがる説明文の断片を連結する。
- 5. **DESCRIPTION_OVERRIDES** により、抽出できない既知の箇所を補う。
- 6. 一致するLHMの定義を更新し、抽出できなかった項目と、更新後も定義が空の項目を分けて報告する。

関数及びデータ

項目	責務
clean_cell 、 clean_description 、 clean_identifier	PDFセルを正規化する。
extract_descriptions	識別子と説明の対応を抽出する。
DESCRIPTION_OVERRIDES	PDF抽出で不足する既知の説明を補う。
read_rows 、 write_rows	LHMを読み込み、書き戻す。
main	抽出した定義を適用し、未解決行を報告する。

終了状態及び依存関係

BG又はBTのいずれかに空の定義が残った場合だけ **1** を返します。**pdfplumber** が必要です。説明を抽出できなくても、LHMに既存の値が入っている場合は報告のみとし、失敗とはしません。

8.4.15 update_lhm_syntax_sequence_from_ubl_xsd.py

役割

展開済みのOASIS UBL 2.1 XML Schemaに定義された実際の子要素順から、LHMの **syntax_sequence** を設定します。

コマンド

```
& $python .\tools\update_lhm_syntax_sequence_from_ubl_xsd.py INPUT.csv `
--output OUTPUT.csv `
--schema-root UBL-2.1\xsd `
[--encoding utf-8-sig]
```

このプログラムはUBLスキーマをダウンロードしません。

処理

1. **SchemaIndex.load_directory** がすべてのXSDを再帰的に読み込む。
2. グローバル要素、名前付き複合型、インラインのシーケンス及び拡張シーケンスを、名前空間とローカル名の組で索引化する。
3. XPath述語及び末端属性を、構造ステップから除去する。
4. **syntax_sequence_for_xpath** がUBL要素宣言をたどり、文書ルートの **0000** から始めて、各子要素の1始まりの位置を4桁で記録する。
5. 末端属性は **@name** の形式で付加する。
6. 解決できた構文順で行を並べ替える。解決できない行は、解決済みの行の後ろに元の順序で残す。
7. 列が存在しない場合は、**sequence** の直後に追加する。

データ構造及び関数

項目	責務
ElementDecl	要素のQName及び任意の型QNameを保持する。
SchemaIndex	XSDの要素、型及び子要素シーケンスを索引化する。
clean_xpath_step 、 split_xpath_steps	XPathの構造ステップを解析する。
split_terminal_attribute 、 xpath_steps	末端属性を分離する。
step_qname	UBL接頭辞を名前空間へ解決する。
syntax_sequence_for_xpath	一つのXPathについてスキーマ上の順序をたどる。
sort_key	解決済みレコードを未解決レコードより前に並べる。
update_lhm	スキーマを読み込み、全行を更新して出力する。
main	引数を解析し、解決件数と総件数を報告する。

検証及び制約

解決できないXPathには空の構文順を設定しますが、終了状態 **1** にはしません。解決件数を確認し、**tests/test_ubl_schema_child_order.py** を実行してください。ダウンロードしたスキーマ式は、ライセンス及びリポジトリ上の役割から明示的に登録が必要な場合を除き、キャッシュ又は生成物用ディレクトリに保存してください。

8.5 推奨保守手順

ソースモデルを変更する場合は、次の順序を推奨します。

1. 編集可能なソースCSV又は管理対象のカバレッジデータを更新する。
2. LHMを構築又は拡張する。
3. 必要に応じて、セマンティックパス、クラス名及び要素名を正規化する。
4. UBLスキーマから構文順を更新する。
5. EN 16931 表2の順序で行を並べ替える。
6. クラス／要素及び対象範囲の検査を実行する。
7. タクソノミを生成して検証する。
8. ラウンドトリップ生成物を再構築し、検証する。

結合定義のチュートリアル又は試作を行う場合は、次の順序を推奨します。

1. **build_syntax_binding.py** で構文結合を生成する。
2. 生成された既定値及び多重度を確認する。
3. **tools/tutorial/syntax_binding_sample.py** で動作を確認する。
4. レビュー済みの結合定義を **specs/bindings/** へ移す。
5. PoC成果物の生成には、運用用の **src/** 配下の変換プログラムと回帰テストを使用する。

8.6 検証マトリクス

変更箇所	最低限実行する検査
LHMの識別子又は階層	test_lhm_semantic_paths.py 、 test_lhm_hierarchical_csv_layout.py
クラス名又は要素名	check_lhm_class_element.py
UBLの構文順	test_ubl_schema_child_order.py
タクソノミ生成	test_xbrlgl_generator_uadc_lhm.py 、 validate_taxonomy.py
構造化CSV又はメタデータ	test_syntax_binding.py 、 test_xbrl_csv_metadata_arelle.py
逆方向変換	test_syntax_binding_reverse.py 、 test_roundtrip_xml_ubl_schema.py
Phase 2結合	出力対象別のADSテスト及びセマンティック結合テスト

8.7 依存関係

多くのツールはPython標準ライブラリだけを使用します。追加の依存関係は次のとおりです。

依存関係	使用箇所
pypdf	audit_en16931_coverage.py
pdfplumber	update_lhm_definitions_from_pdf.py
ブラウザのJavaScript API	psv_viewer.html
Arele (外部検証)	生成したタクソノミ及びメタデータの検証テスト

タクソノミ生成プログラムはローカルのスキーマ資源を使用し、ダウンロードは行いません。ラウンドトリップ生成プログラムは、そのプログラムを起動したのと同じPython実行環境を使用し、リポジトリ内のスクリプトを呼び出します。

8.8 保守規則

- この文書のツール一覧を **tools/** の内容と常に一致させる。
- 引数、関数、生成ファイル、依存関係又は終了動作を変更した場合は、該当する小節も更新する。
- チュートリアル用変換プログラムを、運用用変換プログラムとして記述しない。
- 入力ファイルを直接変更する動作及びディレクトリを消去する動作は明示する。
- Markdownを編集した後は、VSCode Markdown PDFでこの文書のPDFを再生成する。
- 設定済みの余白を使用する。上下 **20mm**、左右 **18mm**。

9. LHM生成の概要

LHM生成文書

この節では、EN 16931請求書LHMの生成方法及びレビュー方法を説明します。

ファイル

- **program_specification.md** : LHM生成プログラムの動作、入力レイアウト、出力レイアウト、セマンティックパス規則、要素名規則及び **lhm_level** の算出方法を定義する。
- **user_guide.md** : LHM CSVを再生成し、出力を確認し、一般的な問題を解決するための運用手順を示す。

関連ディレクトリ

- **../.. /specs/lhm/source/** : 管理されたLHM変更に使用する編集可能なソースCSV。
- **../.. /specs/lhm/** : 変換及びタクソノミ生成で使用する、生成済み又は現行のLHM CSV。
- **../.. /tools/** : ソースからの再構築、対象範囲の検査、クラス／要素の正規化及び構文順設定など、LHM保守用スクリプト。

人によるレビューに使用するExcelブックはローカルだけに保存し、Gitの管理対象外とします。

10. LHM生成プログラム仕様

プログラム仕様 : LHM生成

1. 目的

この節では、UADC概念実証（PoC）で使用するEN 16931請求書の論理階層モデル（LHM）CSVを生成し、検証するプログラムの仕様を示します。

この節に記載するすべてのパスは、リポジトリをpush又はcloneした後の **UADC_PoC** 作業ディレクトリを基準とします。

2. 対象範囲

現在の基準は、EN 16931-1の請求書セマンティックモデルです。OpenPeppol BIS Billingは、後続段階でCIUS／プロファイルの上書きとして扱います。

管理対象のソース：

```
specs/lhm/source/EN16931_CIUS_Invoice_LHM_Source.csv
```

生成するLHM：

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

3. 構成要素

3.1 ソースからLHMを生成するプログラム

プログラム：

```
tools/build_lhm_from_source.py
```

責務：

- **init-source** により、既存のLHM CSVから編集可能なソースCSVを作成する。
- **build** により、編集可能なソースCSVから正規化済みのLHM CSVを生成する。
- Business Termの値からlowerCamelCaseConcatenated形式のセマンティックパスセグメントを生成する。
- 最も近い親BGのBusiness Termを単数形にして **class_term** を生成する。
- 構造化CSV及びxBRL-CSVタクソノミのモデル化に使用する **lhm_level** を算出する。
- 上書き値がない場合、重複しないUpperCamelCase形式の **element** を生成する。
- 手動上書き列を保持する。

3.2 クラス名及び要素名の正規化プログラム

プログラム：

```
tools/normalize_lhm_class_element.py
```

責務：

- **class_term** を正規化する。
- **element** を正規化する。
- 生成する要素名の一意性を確保する。

規則：

- BG行の **class_term** には、そのBG自身のBusiness Termを単数形にした値を使用する。
- BT行の **class_term** には、最も近い親BGのBusiness Termを単数形にした値を使用する。
- **element** は **semantic_path** から生成する。
- **element** は大文字で始める。
- セマンティックパスの末尾セグメントが重複する場合は、重複しない最短のパス末尾部分を使用する。

3.3 クラス名及び要素名の検査プログラム

プログラム：

```
tools/check_lhm_class_element.py
```

責務：

- **class_term** の不一致を報告する。
- **element** の不一致を報告する。
- 不一致がある場合は、0以外の終了コードを返す。

3.4 PDFから定義を更新するプログラム

プログラム：

```
tools/update_lhm_definitions_from_pdf.py
```

責務：

- **pdfplumber** を使い、EN 16931-1 表2のDescriptionセルを抽出する。
- 抽出したBT／BG識別子に対応するLHMの **definition** 列を更新する。
- PDF抽出時にセルが分割されることが分かっている行について、組み込みの上書きを適用する。

- 定義が空のまま残った行を報告する。

このユーティリティは、指定したLHM CSVを更新します。

3.5 UBL構文順更新プログラム

プログラム：

```
tools/update_lhm_syntax_sequence_from_ubl_xsd.py
```

責務：

- 展開済みのOASIS UBL 2.1 XSDファイルを読み込む。
- LHMの各XPathを、UBL Invoiceスキーマのシーケンスに照らして解決する。
- XML要素順の検査に使用できる **syntax_sequence** を書き込む。
- ダウンロードしたUBLスキーマ式は、通常 **out/cache** 配下など、バージョン管理対象外の場所に保持する。

4. 編集可能なソースCSV

列：

```
sequence
syntax_sequence
id
level
type
cardinality
business_term
description
usage_note
req_id
semantic_data_type
path
xpath
semantic_path_override
class_term_override
element_override
label_local
definition_local
source_ref
adjustment_note
```

主な列：

- **sequence**：EN 16931-1 表2の順序。

- **syntax_sequence** : OASIS UBL 2.1 XSDから設定した場合の、UBL Invoice XMLスキーマ上の順序。
- **id** : **BG-4**、**BT-27** などのEN 16931識別子。
- **level** : 階層レベル。Invoiceは **0**、**BT-1** は **1**。
- **cardinality** : 入力資料上の多重度。..**n** で終わる値は **..*** に正規化する。
- **business_term** : EN 16931のBusiness Term。
- **description** : EN 16931のDescription。LHMの **definition** になる。
- **path** : 親BGを特定するための、スラッシュ区切り識別子パス。
- **xpath** : 利用可能な場合に設定するUBL Invoiceの構文結合参照パス。
- **semantic_path_override** : 任意の完全なセマンティックパス上書き。
- **class_term_override** : 任意のクラス名上書き。
- **element_override** : 任意の要素名上書き。

5. 生成するLHM CSV

列 :

```
sequence
syntax_sequence
level
lhm_level
type
identifier
name
datatype
multiplicity
domain_name
definition
module
class_term
id
path
semantic_path
label_local
definition_local
element
xpath
```

対応規則 :

- **level** は、EN 16931／LHMの論理階層を保持する。
- **lhm_level** は、構造化CSV及びxBRL-CSVタクソノミ生成で使用する実効階層である。
- **BG-ROOT** の **lhm_level** は **0** とする。
- 多重度が **0..*** 又は **1..*** のBGは、**lhm_level** を持つ最も近い祖先BGの値に **1** を加えた値を **lhm_level** とする。

- 多重度が **0..1** 又は **1..1** のBGは、**BG-ROOT** を除き **lhm_level** を空欄とする。
- BTは、**lhm_level** を持つ最も近い祖先BGの値に **1** を加えた値を **lhm_level** とする。
- **lhm_level** が空欄のBGもセマンティックモデルには保持するが、構造化CSVのディメンション列又はxBRL-CSVのディメンション概念としては出力しない。
- **name** は **business_term** からコピーする。
- **syntax_sequence** はソースCSVからコピーするか、UBL構文順更新プログラムで設定する。
- **datatype** は **semantic_data_type** からコピーする。
- **multiplicity** は **cardinality** からコピーする。
- **definition** は **description** からコピーする。
- **module** は現在 **en16931** とする。
- **semantic_path** には、**semantic_path_override** 又は生成したパスを使用する。
- **element** には、**element_override** 又は生成した一意なUpperCamelCase名称を使用する。

セマンティックパス規則：

- Business TermをlowerCamelCaseConcatenated形式のパスセグメントへ変換する。
- セマンティックパスは **\$.invoice** から始める。
- EN 16931-1に定義がないため、**BG-0** は生成しない。

多重度規則：

- LHMの **multiplicity** は、**0..1**、**0..**、**1..1** 又は **1..** のいずれかとする。
- 入力資料の **0..n** 及び **1..n** は、それぞれ **0..*** 及び **1..*** に正規化する。
- その他の多重度は、LHM生成時にエラーとする。

要素名の一意性規則：

1. 先頭の **\$.** を除去した後、**semantic_path** をパスセグメントに分割する。
2. 最後のセグメントを最初の候補とする。
3. 候補セグメント又はパス末尾部分をUpperCamelCaseへ変換する。
4. その名称が一意であれば、**element** として使用する。
5. 他の行と重複する場合は、左側のセグメントを一つ追加して再試行する。
6. 重複しない最短のセマンティックパス末尾部分が見つかるまで繰り返す。
7. どの末尾部分も一意でない場合は、最後の代替としてハイフンを除いた行識別子を付加する。

例：

```
$.invoice.precedingInvoiceReference.precedingInvoiceReference
```

最後のセグメントだけでは **PrecedingInvoiceReference** となり、BG行の名称と重複します。そのため、生成プログラムは末尾部分を一段拡張し、次の名称を生成します。

```
PrecedingInvoiceReferencePrecedingInvoiceReference
```

例：

```
BT-1 Invoice number
semantic_path = $.invoice.invoiceNumber
class_term = Invoice
element = InvoiceNumber
level = 1
```

6. 検証規則

LHMの検査では、次を確認します。

- セマンティックパスのセグメントがlowerCamelCaseConcatenated形式である。
- **BG-0** が存在しない。
- **BG-ROOT** がレベル **0** のInvoiceを表す。
- **BT-1** がレベル **1** の **\$.invoice.invoiceNumber** である。
- LHMの要素名が一意である。
- 多重度が **0..1**、**0..**、**1..1** 及び **1..** に限定されている。
- 階層型CSV出力で、BGディメンション列が左詰めに配置されている。
- Seller、Buyerなどの非繰返しBGがディメンション列として出力されない。
- Invoice Lineなどの繰返しBGがディメンション列として出力される。

7. 依存関係

必須：

- Python 3.10以降。

任意：

- **pdfplumber**。 **tools/update_lhm_definitions_from_pdf.py** を使用する場合だけ必要。

8. 対象外

LHM生成プログラムは、次を行いません。

- 生成した **out/** ファイルをGitHubへ公開する。
- OpenPeppol BIS Billingのプロファイル制約を完全にモデル化する。
- XBRLタクソノミ出力を検証する。

11. LHM生成ユーザーガイド

ユーザーガイド：LHM生成

1. 作業ディレクトリ

UADC_PoC ディレクトリでコマンドを実行します。

```
cd UADC_PoC
```

以下のパスはすべて、このディレクトリを基準とします。

Windowsのローカル環境で使用するPythonコマンドを設定します。

```
$python = 'python'
```

2. LHMソースの編集

次のファイルを編集します。

```
specs/lhm/source/EN16931_CIUS_Invoice_LHM_Source.csv
```

通常編集する列：

- **description**
- **path**
- **xpath**
- **semantic_path_override**
- **class_term_override**
- **element_override**
- **label_local**
- **definition_local**
- **adjustment_note**

生成値をそのまま使用できる場合は、上書き列を空欄にします。

生成LHMでは、次の多重度だけを使用します。

```
0..1  
0..*
```

```
1..1
1..*
```

入力資料で **0..n** 又は **1..n** が使用されている場合、LHM生成プログラムはそれぞれ **0..*** 又は **1..*** に正規化します。

生成LHMには、**level** と **lhm_level** の両方があります。

- **level** はEN 16931／LHMの論理階層を保持する。
- **lhm_level** は構造化CSV及びxBRL-CSVタクソノミ生成で使用する実効階層である。
- 多重度が **0..1** 又は **1..1** のBGは通常 **lhm_level** が空欄であり、ディメンションとして出力されない。
- それらのBG配下にあるBTは、**lhm_level** を持つ最も近い祖先BGの値に **1** を加えた値を使用する。
- 多重度が **0..*** 又は **1..*** の繰返しBGは、自身の **lhm_level** を持ち、ディメンションとなる。

3. LHM CSVの生成

次を実行します。

```
& $python .\tools\build_lhm_from_source.py build `
.\specs\lhm\source\EN16931_CIOUS_Invoice_LHM_Source.csv `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv
```

想定される出力：

```
Wrote generated LHM CSV: specs\lhm\EN16931_CIOUS_Invoice_LHM.csv
```

4. 任意：ソースCSVの初期化

既存のLHM CSVから、新しい編集可能なソースCSVを作成する必要がある場合だけ使用します。

```
& $python .\tools\build_lhm_from_source.py init-source `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv `
.\specs\lhm\source\EN16931_CIOUS_Invoice_LHM_Source.csv
```

このコマンドはソースCSVを書き換えます。慎重に使用してください。

5. 任意：クラス名及び要素名の正規化

原則として、ソースCSVからLHMを再生成してください。生成済みLHM CSVを直接編集した場合は、次のコマンドで正規化します。

```
& $python .\tools\normalize_lhm_class_element.py `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv
```

6. 任意 : UBL構文順の設定

公式のOASIS UBL 2.1パッケージをダウンロードして、次のようなローカルのバージョン管理対象外ディレクトリへ展開します。

```
out/cache/UBL-2.1
```

次に、UBL Invoiceスキーマから **syntax_sequence** を設定します。

```
& $python .\tools\update_lhm_syntax_sequence_from_ubl_xsd.py `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv `
--schema-root .\out\cache\UBL-2.1\xsd `
-o .\out\cache\EN16931_CIOUS_Invoice_LHM.syntax_sequence_check.csv
```

生成ファイルを使ってXMLスキーマ上の順序を確認します。EN 16931の **sequence** 列は表2のセマンティックな順序を保持し、**syntax_sequence** はXML向け検査及び逆方向出力の並べ替えに使用するUBL XML順を表します。

7. 任意 : PDFから定義を更新

EN 16931-1 PDFを利用でき、**pdfplumber** がインストールされている場合は、次を実行します。

```
& $python .\tools\update_lhm_definitions_from_pdf.py `
"<path-to-EN16931-1-pdf>" `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv `
--first-page 43 `
--last-page 75
```

このコマンドは、コマンドラインで指定したCSVファイルだけを更新します。

8. LHMの検査

セマンティックパスの検査 :

```
& $python .\tests\test_lhm_semantic_paths.py
```

クラス名及び要素名の検査：

```
& $python .\tools\check_lhm_class_element.py `
.\specs\lhm\EN16931_CIUS_Invoice_LHM.csv
```

階層型CSVレイアウトの検査：

```
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

9. トラブルシューティング

要素名が重複する

ソースCSVからLHMを再生成してください。要素名は、**semantic_path** の末尾から重複しない最短の部分を使用して生成されます。意図的に同じ名称を使う必要がある場合は、ソースCSVの **element_override** を設定してください。

クラス名が正しくない

ソースCSVの **path** 列を確認してください。BT行は、そのパスに含まれる最も近い親BGを使用します。

セマンティックパスが正しくない

ソースCSVの **semantic_path_override** を設定し、LHMを再生成してください。

PDFから説明を取得できない

PDFのページ範囲を確認してください。PDFの表抽出ではセルが分割される場合があるため、一部の行はソースCSVの **description** を手動で設定する必要があります。

12. タクソノミ生成の概要

タクソノミ生成文書

この節では、LHMからxBRL-CSVタクソノミを生成する方法を説明します。

ファイル

- **program_specification.md**：タクソノミ生成プログラムの入力、出力ファイル、xBRL-CSV上の制約、ディメンション関係及び検証規則を定義する。
- **user_guide.md**：ローカルテスト及びArelleを使用してタクソノミを生成し、確認するためのコマンド例を示す。

関連ディレクトリ

- `../tools/taxonomy/` : UADC互換のローカルタクソノミ生成プログラム及びGL汎用スキーマひな型。
- `../specs/lhm/` : タクソノミの入力として使用するLHM CSV。
- `../out/taxonomy/` : PoCの検証証跡としてGitで管理する生成済みタクソノミ。

Phase 1では、xBRL-CSVタクソノミのエントリポイントとして **en16931-oim-.xsd** を、ディメンション定義リンクベースとして **en16931-def.xml** を生成します。エントリポイントは、LHM階層をタクソノミ処理系から確認できるよう、EN 16931の表示リンクベースを参照します。**plt-all-.xsd** 又は **-content-.xsd** のタプル/コンテンツスキーマは生成しません。

13. タクソノミ生成プログラム仕様

プログラム仕様 : タクソノミ生成

1. 目的

この節では、UADC概念実証（PoC）で使用するタクソノミ生成プログラムの仕様を示します。

この節に記載するすべてのパスは、リポジトリをpush又はcloneした後のリポジトリルートを基準とします。このPoCで使用するタクソノミ生成プログラムは、リポジトリに含まれています。

2. 対象範囲

入力LHM :

```
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv
```

タクソノミ生成プログラム :

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

出力ディレクトリ :

```
out/taxonomy/
```

出力ディレクトリはローカルの生成物であり、GitHubへpushすることを想定していません。

3. 構成要素

プログラム :

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

責務：

- **csv.DictReader** を使用してUADC LHM CSVの列構成を読み込む。
- UADC LHMのレコードを、XBRL-GL生成プログラムの内部形式へ正規化する。
- モジュールタクソノミのスキーマファイルを生成する。
- xBRL-CSV用ディメンショナルタクソノミスキーマを生成する。
- ラベル、表示リンクベース、定義リンクベース、JSONメタデータ及びCSVスケルトンを生成する。

生成プログラムは、UADC LHM CSVの見出しを直接受け付けます。

4. 入力仕様

生成プログラムは、次の生成済みLHM CSVを入力とします。

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

必須列には、次が含まれます。

```
sequence
level
lhm_level
type
name
datatype
multiplicity
definition
module
class_term
id
path
semantic_path
element
xpath
```

主な解釈規則：

- **module** はモジュールの名前空間接頭辞になる。現在は **en16931**。
- **element** は概念名になる。
- **type** により、ディメンションモデル化において行を項目又はグループのどちらとして扱うかを決定する。
- **multiplicity** はディメンションモデル化に使用する。

- **level**、**path** 及び **semantic_path** はLHMの論理階層を保持する。
- **lhm_level** はxBRL-CSVタクソノミ生成で使用する実効階層を定義する。
- **lhm_level** が空欄のBG行は意味上のグループとして保持するが、ハイパーキューブ、ディメンション又はプライマリ項目の概念としては出力しない。
- BT行は、**lhm_level** を持つ最も近い祖先BGのキューブへ割り当てる。

使用できる **multiplicity** は、**0..1**、**0..**、**1..1** 及び **1..** です。LHM生成段階で、**0..n**、**1..n** などの入力値を正規化してから、タクソノミを生成します。

5. タクソノミ出力

EN 16931 PoCでは、次を生成します。

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label-ja.xml
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/en16931-def-2026-07-05.xml
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
```

5.1 モジュールスキーマ

ファイル名形式：

```
out/taxonomy/<module>/<module>-<version>.xsd
```

このPoCでは、次のファイルです。

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
```

責務：

- LHMのField行に対応する項目概念を宣言する。
- **substitutionGroup="xbrli:item"** の要素に **xbrli:periodType="instant"** を設定する。
- GL汎用型を使用する場合、**../gen/gl-gen-.xsd** からGL汎用型スキーマをimportする。

5.2 生成しないタブル/コンテンツスキーマ

この構造化CSV PoCでは、XBRL 2.1タブル用タクソノミエントリポイント及びタブル/コンテンツスキーマを生成しません。特に、次のファイルは定義しません。

- **out/taxonomy/plt/plt-all-.xsd**
- **out/taxonomy/plt/-content-.xsd**

5.3 GL汎用型スキーマ

ファイル名形式：

```
out/taxonomy/gen/gl-gen-<version>.xsd
```

責務：

- **gen:amountItemType**、**gen:emailAddressItemType** など、再利用可能なGL項目型を提供する。
- 生成タクソノミと同じバージョン名前空間を使用する。
- モジュールスキーマから **../gen/gl-gen-.xsd** として参照される。

5.4 xBRL-CSVタクソノミスキーマ

ファイル名形式：

```
out/taxonomy/plt/en16931-oim-<version>.xsd
```

責務：

- **xbrldt:hypercubeItem** を使用して **h_*** ハイパーキューブ概念を定義する。
- **xbrldt:dimensionItem** を使用して **d_*** ディメンション概念を定義する。
- **xbrli:item** を使用して **p_*** プライマリ項目概念を定義する。
- 型付きディメンションのドメイン要素 **_v** を定義する。
- xBRL-CSV定義リンクベースで使用するロール型を定義する。
- **en16931-def.xml**、モジュールラベル及びモジュール表示リンクベースを参照する。
- OIMのプライマリ項目及びディメンションにEN 16931のtarget namespaceを使用する。

制約：

- **en16931-oim** は、タプル対応の **complexType** を定義してはならない。
- **en16931-oim** は、**xbrli:tuple** 概念を定義してはならない。
- **en16931-oim** は、モジュールのコンテンツスキーマをimportしてはならない。

5.5 定義リンクベース

ファイル名形式：

```
out/taxonomy/plt/en16931-def-<version>.xml
```

責務：

- **en16931-oim** のロール型及びディメンション概念を参照する。
- プライマリ項目、ハイパーキューブ及びディメンションを結び付ける。
- xBRL-CSV用のディメンション関係により、BG間の階層を表す。

ディメンション関係の定義リンクベースロケータは、**en16931-oim** を参照します。

5.6 表示階層

OIMエントリポイントは、**out/taxonomy/en16931/en16931--presentation.xml** を参照します。表示階層はLHMの親子ツリーから生成します。クラス／BG行は **en16931-oim** にある対応する **p_en16931_*** プライマリ項目へ解決し、BT／ファクト行はEN 16931モジュールスキーマの概念へ解決します。これにより、タプル概念を導入せずに、ArelleのPresentationビューでLHM階層を確認できます。

6. 検証規則

タクソノミの回帰検査では、次を確認します。

- **en16931-oim** にハイパーキューブ、ディメンション及びプライマリ項目の定義がある。
- **en16931-oim** に **xbrli:tuple** がない。
- **en16931-oim** に **complexType** がない。
- **en16931-oim** が **en16931-content** をimportしていない。
- OIMエントリポイントから表示リンクベースを検出できる。
- すべての表示ロケータが、OIMプライマリ項目又はモジュールファクトへ解決される。
- **plt-all** が生成されない。
- **en16931-content** が生成されない。
- モジュールスキーマが **../gen/gl-gen-.xsd** をimportする。
- **en16931-oim** に生成した **xbrli:item** 要素に **xbrli:periodType="instant"** が設定されている。

7. 依存関係

必須：

- Python 3.10以降。

8. 対象外

タクソノミ生成プログラムは、次を行いません。

- LHMソースCSVの更新。

- 生成した **out/** ファイルのGitHubへの公開。
- 外部XBRLプロセッサによる生成DTSの検証。
- 回帰検査を超えるxBRl-CSVインスタンスの検証。

14. タクソノミ生成ユーザーガイド

ユーザーガイド：タクソノミ生成

1. 作業ディレクトリ

UADC_PoC ディレクトリでコマンドを実行します。

```
cd UADC_PoC
```

以下のパスはすべて、このディレクトリを基準とします。タクソノミ生成プログラムは **tools/taxonomy/** に含まれています。

Windowsのローカル環境で使用するPythonコマンドを設定します。

```
$python = 'python'
```

2. 入力

タクソノミ生成プログラムは、次を使用します。

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

ソースモデルを変更した場合は、タクソノミを生成する前にLHMを再生成し、検査してください。

3. タクソノミの生成

タクソノミ回帰スクリプトを実行します。

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

このスクリプトは、次のディレクトリ配下を再生成します。

```
out/taxonomy/
```

4. 生成プログラムの直接実行

タクソノミ生成プログラムを直接実行する場合は、次を使用します。

```
& $python .\tools\taxonomy\xBRLGL_TaxonomyGenerator.py `
.\specs\lhm\EN16931_CIUS_Invoice_LHM.csv `
-b .\out\taxonomy `
-p en16931 `
-n https://example.com/uada/en16931/invoice/2026-07-05 `
-l ja `
-c JPY
```

主なオプション：

- **-b、--base_dir**：出力ディレクトリ。
- **-p、--palette**：生成ファイルに使用するパレット／モジュール接頭辞。
- **-r、--root**：JSONメタデータ及びCSVスケルトン生成に使用するルート要素名。
- **-l、--lang**：ローカルラベルの言語。
- **-c、--currency**：金額ファクトに使用するISO通貨コード。
- **-n、--namespace**：名前空間。末尾10文字をバージョン日付として使用する。
- **-e、--encoding**：入出力の文字エンコーディング。既定値は **utf-8-sig**。
- **-t、--trace**：トレースメッセージを出力する。
- **-d、--debug**：デバッグメッセージを出力する。

5. 出力ファイル

主なファイル：

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label-ja.xml
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/en16931-def-2026-07-05.xml
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
```

意味：

- **en16931/en16931-2026-07-05.xsd**：モジュールの項目概念。
- **gen/gl-gen-2026-07-05.xsd**：../gen/gl-gen-.xsd として参照されるGL汎用項目型スキーマ。
- **plt/en16931-oim-2026-07-05.xsd**：ハイパーキューブ、ディメンション、プライマリ項目並びにラベル、表示関係及び定義関係への参照を含むxBR-CSVタクソノミエントリポイント。
- **plt/en16931-def-2026-07-05.xml**：xBR-CSV用ディメンション定義リンクベース。

- **en16931/en16931-2026-07-05-presentation.xml** : LHMの親子階層。BG/ClassノードはOIMプライマリ項目を、BTノードはモジュールファクトを参照する。

6. タクソノミ分離の確認

ローカルのタクソノミ構造が整合していることを確認します。

```
& $python .\tests\validate_taxonomy.py
```

想定される出力：

```
ok: local taxonomy checks passed
```

en16931-oim にタプル又はコンテンツスキーマ定義が含まれていないことを確認します。

```
Select-String -Path .\out\taxonomy\plt\en16931-oim-2026-07-05.xsd `
  -Pattern 'en16931-content','complexType','xbrli:tuple'
```

想定される結果：一致なし。

en16931-oim にハイパーキューブ、ディメンション及びプライマリ項目の概念が含まれていることを確認します。

```
Select-String -Path .\out\taxonomy\plt\en16931-oim-2026-07-05.xsd `
  -Pattern
  'xbrldt:hypercubeItem','xbrldt:dimensionItem','substitutionGroup="xbrli:item"'
```

モジュールスキーマがGL汎用スキーマをimportしていることを確認します。

```
Select-String -Path .\out\taxonomy\en16931\en16931-2026-07-05.xsd `
  -Pattern '../gen/gl-gen-2026-07-05.xsd'
```

plt-all 及び **en16931-content** が生成されていないことを確認します。

```
Test-Path .\out\taxonomy\plt\plt-all-2026-07-05.xsd
Test-Path .\out\taxonomy\plt\en16931-content-2026-07-05.xsd
```

想定される結果：**False**。

7. Arelleによるタクソノミ検証

Arelleがインストールされている場合は、次を実行します。

```
& arelleCmdLine.exe `
  --file .\out\taxonomy\plt\en16931-oim-2026-07-05.xsd `
  --validate
```

想定される出力には、次が含まれます。

```
[info] validated
```

8. 回帰検査の実行

次を実行します。

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

想定される出力には、次が含まれます。

```
ok: XBRL-GL generator accepted UADC LHM CSV
```

9. トラブルシューティング

Pythonが見つからない

Pythonのフルパスを使用し、次で確認します。

```
& $python --version
```

en16931-oim にタプル又はコンテンツ定義が含まれる

現在の生成プログラムで再生成します。

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

現在の設計はxBRl-CSV専用です。ハイパーキューブ、ディメンション及びプライマリ項目の定義は **en16931-oim** に保持しますが、XBRL 2.1のタプル概念は生成しません。

タクソノミの日付が想定と異なる

-n 名前空間オプションを確認してください。生成プログラムは名前空間の末尾10文字をバージョン日付として使用します。